

RECORD Y DTO

En el ecosistema Java

Una explicación simple para construir APIs limpias en Spring Boot sin confundir dominio, DTO y persistencia.



Capítulo complementario · Backend con Spring Boot

El problema que resuelven Record y DTO

Sin DTO

El controller expone cualquier objeto interno. Se mezcla lo que usa la API con lo que usa el dominio o la base de datos.

Sin record

Escribimos clases repetitivas para transportar datos: atributos, constructor, getters, equals, hashCode y toString.

Con record + DTO

Definimos contratos claros, inmutables y compactos para entrada y salida HTTP.

Idea central: un DTO dice qué datos entran o salen de la API. Un record es una forma moderna y compacta de escribir ese DTO en Java.

¿Qué es un record en Java?

```
1 public record ClienteResponse (
2     Long id,
3     String nombreCompleto,
4     String email,
5     boolean activo
6 ) {}
```

No se accede con getEmail(). En un record se usa email().

Un record es una clase especial

Sirve para representar datos de manera compacta.

Es inmutable por diseño

Sus componentes son finales. No se modifica como un objeto mutable clásico.

Genera código automáticamente

Constructor, getters con nombre del componente, equals, hashCode y toString.

Ideal para transportar datos

Perfecto para DTOs, respuestas, comandos simples y proyecciones.

Clase tradicional vs record

Clase tradicional

```
1 public class ClienteResponse {  
2     private final Long id;  
3     private final String em ail;  
4  
5     public ClienteResponse(Long id, String em ail) {  
6         this.id = id;  
7         this.em ail = em ail;  
8     }  
9  
10    public Long getId() { return id; }  
11    public String getEm ail() { return em ail; }  
12 }
```

Record

```
1 public record ClienteResponse(  
2     Long id,  
3     String em ail  
4 ) {}
```

El record elimina ceremonia. Pero no elimina la necesidad de pensar bien el contrato.

Regla para estudiantes: menos código no significa menos diseño.

¿Qué es un DTO?



Request DTO

Representa lo que el cliente envía.
Ejemplo: crear un cliente.

Response DTO

Representa lo que la API responde.
Ejemplo: cliente creado.

DTO ≠ Entidad

No debe ser automáticamente igual a la tabla ni igual al dominio.

DTO significa Data Transfer Object: objeto para transferir datos entre límites, especialmente entre API y cliente.

Ejemplo ParkFlow: Request DTO y Response DTO

```
1 public record CrearClienteRequest(  
2     @NotBlank String nombreCompleto,  
3     @NotBlank String documento,  
4     @Email String email,  
5     String telefono  
6 ) {}
```

Entrada

El request puede exigir datos obligatorios y validaciones con Jakarta Validation.

```
1 public record ClienteResponse(  
2     Long id,  
3     String nombreCompleto,  
4     String documento,  
5     String email,  
6     String telefono,  
7     boolean activo  
8 ) {}
```

Salida

La respuesta puede incluir ID generado, estado activo y campos que el cliente necesita ver.

No todos los campos de entrada tienen que aparecer en salida. No todos los campos de la tabla tienen que exponerse en la API.

Record + Validación: una combinación poderosa

```
1 public record CrearVehiculoRequest(  
2     @NotNullLong clienteId,  
3     @NotBlank @Pattern(regexp = "[A-Z0-9-]{5,10}") String  
4     placa,  
5     @NotBlank String marca,  
6     @NotBlank String modelo,  
7     @NotNullTipoVehiculo tipo  
8 ) {}
```

@NotNull

El campo debe llegar en el JSON.

@NotBlank

El texto no puede venir vacío.

@Pattern

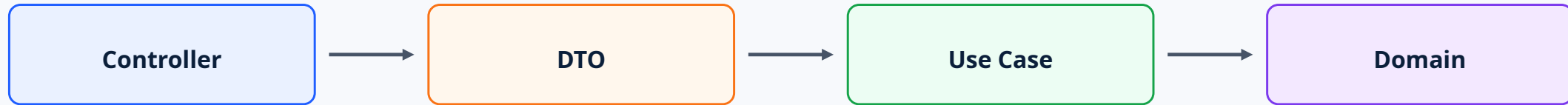
El texto debe cumplir un formato.

@Email / @Size

Reglas frecuentes para contratos HTTP.

Validación de formato no es lo mismo que regla de negocio. Placa vacía = formato. Placa duplicada = negocio.

Dónde vive cada cosa en arquitectura hexagonal



DTO

Contrato externo: lo que entra y sale por HTTP.

Dominio

Reglas del negocio y lenguaje propio del sistema.

JPA Entity

Representación de la tabla para persistencia.

Una API limpia no expone directamente entidades JPA. El DTO protege el contrato y evita acoplar HTTP a la base de datos.

Mapper: el puente entre mundos

```

1 public class ClienteWebMapper {
2
3     public Cliente toDomain(CrearClienteRequest request) {
4         return Cliente.nuevo(
5             request.nombreCompleto(),
6             request.documento(),
7             request.email(),
8             request.telefono()
9         );
10    }
11
12    public ClienteResponse toResponse(Cliente cliente) {
13        return new ClienteResponse(
14            cliente.getId(),
15            cliente.getNombreCompleto(),
16            cliente.getDocumento(),
17            cliente.getEmail(),
18            cliente.getTelefono(),
19            cliente.isActivo()
20        );
21    }
22 }

```

Entrada

Request DTO → Domain

Salida

Domain → Response DTO

Regla clave

No hacer conversiones complejas dentro del controller.

Errores comunes que debes evitar

Error 1: exponer la entidad JPA

`return clienteJpaEntity; // acopla API a la base de datos`

Error 4: creer que record es siempre mejor

Si necesitas comportamiento mutable complejo, quizás no corresponde.

Error 2: usar el mismo DTO para todo

Crear, actualizar, listar y detalle no siempre necesitan los mismos campos.

Error 5: no diferenciar null y vacío

null, "" y dato inválido no significan lo mismo.

Error 3: meter reglas de negocio en el DTO

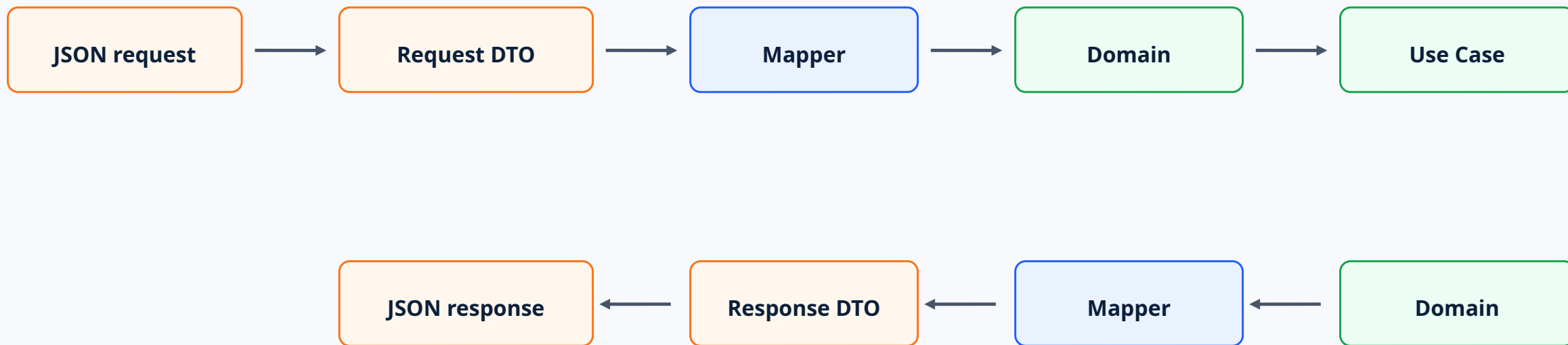
El DTO valida formato; el caso de uso valida negocio.

Error 6: copiar DTOs del profesor

Cada proyecto debe reflejar su propio dominio y sus propias reglas.

Pregunta de defensa: ¿por qué este campo está en el DTO de salida pero no en el DTO de entrada?

Cómo explicarlo en una API real



Entrada

Validar contrato HTTP.

Centro

Ejecutar reglas de negocio.

Salida

Responder sólo lo necesario.

Checklist para tus estudiantes

- Puedo explicar qué es un record sin decir solamente "es una clase corta".
- Puedo diferenciar Request DTO, Response DTO, dominio y entidad JPA.
- Sé por qué un DTO protege el contrato HTTP.
- Uso validaciones de formato en el DTO y reglas de negocio en el caso de uso.
- No expongo entidades JPA directamente desde el controller.
- Puedo justificar cada campo que entra y sale de la API.

Frase final para clase: record es una herramienta del lenguaje; DTO es una decisión de diseño de arquitectura.